



Tile Sistemi Temelleri

Tile Kavramı	
Terim	Anlam
Tile	Sabit boyutlu hücre
Tileset	Tile'ları içeren PNG
Grid	Satır/sütun ızgarası
Tile ID	Tile türü (int)
Harita	2B int listesi

2B Harita Tanımı	
<pre>TILE = 32 HARITA = [[2, 2, 2, 2], [2, 1, 1, 2], [2, 1, 0, 2], [2, 2, 2, 2],]</pre>	
# Eriş: HARITA[row][col]	

Tileset Kesme	
<pre>tileset = pygame.image.load("tileset.png").convert_alpha() tiles = [] for r in range(2): for c in range(4): rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE) tiles.append(tileset.subsurface(rect))</pre>	

Grid ↔ Piksel	
Dönüşüm	Formül
Piksel → Grid	col = px // TILE row = py // TILE
Grid → Piksel	x = col * TILE y = row * TILE

Harita Yükleme

CSV Yükleme	
<pre>import csv from pathlib import Path def harita_yukle(yol): with open(yol, newline="") as f: return [[int(x) for x in row] for row in csv.reader(f) if row] yol = Path(__file__).parent harita = harita_yukle(yol / "harita.csv")</pre>	

pytmx ile .tmx	
<pre>from pytmx.util_pygame \ import load_pygame tmx = load_pygame("harita.tmx") # tmx.width, tmx.height # tmx.tilewidth for layer in tmx.visible_layers: if hasattr(layer, "tiles"): for x, y, img in layer.tiles(): ekran.blit(img, (x*TILE, y*TILE))</pre>	

CSV vs pytmx	
Özellik	Seçim
Basit prototip	CSV
Çok katman	pytmx
Görsel editor	Tiled
Ek bağımlılık yok	CSV

Kamera Sistemi

Temel Camera	
<pre>class Camera: def __init__(self, gen, yuk, lerp=0.1): self.offset = \ pygame.Vector2(0, 0) self.gen = gen self.yuk = yuk self.lerp = lerp def apply(self, rect): return rect.move(-int(self.offset.x), -int(self.offset.y)) def update(self, hedef): ist = pygame.Vector2(hedef.centerx - self.gen//2, hedef.centery - self.yuk//2) self.offset += \ (ist - self.offset) * self.lerp</pre>	

Clamp	
<pre>def _clamp(self): mx = max(0, self.dunya_gen - self.gen) my = max(0, self.dunya_yuk - self.yuk) self.offset.x = max(0, min(self.offset.x, mx)) self.offset.y = max(0, min(self.offset.y, my))</pre>	

Lerp Faktörleri	
t	His
0.02	Lazy, keşif
0.1	Doğal (önerilen)
0.3	Reaktif
1.0	Anlık (lerp yok)

Platform Fiziği

Yatay + Dikey Ayır	
<pre># YATAY self.rect.x += self.vx for k in katilar: if self.rect.colliderect(k): if self.vx > 0: self.rect.right = k.left elif self.vx < 0: self.rect.left = k.right # DIKEY self.vy += 0.5 self.rect.y += int(self.vy) self.yerde = False for k in katilar: if self.rect.colliderect(k): if self.vy > 0: self.rect.bottom = k.top self.yerde = True elif self.vy < 0: self.rect.top = k.bottom self.vy = 0</pre>	

Zıplama	
<pre>def zipla(self): if self.yerde: self.vy = -10</pre>	

Seviye Tile Grupları	
<pre>def gruplar(harita): katilar = [] tehlikeler = [] altinlar = [] for r, s in enumerate(harita): for c, t in enumerate(s): rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE) if t == 2: katilar.append(rect) elif t == 5: tehlikeler.append(rect) elif t == 4: altinlar.append(rect) return (katilar, tehlikeler, altinlar)</pre>	

Tehlike + Spawn	
<pre>for t in tehlikeler: if oyuncu.rect.colliderect(t): oyuncu.can -= 1 oyuncu.rect.topleft = \ oyuncu.spawn oyuncu.vx = oyuncu.vy = 0 break # çift zarar onle</pre>	

Toplanabilir

```
#[:] kopyasi üzerinde dolasi!  
for a in altinlar[:]:  
    if oyuncu.rect.colliderect(a):  
        altinlar.remove(a)  
        skor += 10
```

Hata Önleme

Hata	Çözüm
Liste silme	<code>liste[:]</code>
Tunnelling	$Hız \leq TILE$
Negatif grid	<code>max(0, ...)</code>
Köşe takılma	Yatay/dikey ayır